

EXPERIMENT - 6

AIM

To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling, implement triggers for salary validation and audit logging, and test important edge cases.

OBJECTIVES

- To create a stored procedure for transferring an employee between departments.
- To validate employee and department identifiers before performing a transfer.
- To use transaction control and error handling inside a stored procedure.
- To validate salary values using a trigger before INSERT or UPDATE.
- To maintain an audit log whenever an employee salary is changed.
- To test successful and invalid transfer and salary-update cases.

DATABASE USED

The queries use the CompanyDB database and the Employee-Department-Project schema. Department names and employee names have been changed from the reference data; IDs remain unchanged so the existing relationships continue to work.

USE CompanyDB;

SOURCE CODE / SQL QUERIES

1. SALARY AUDIT TABLE

Create an audit table to store old and new salary values whenever an employee salary changes.

```
CREATE TABLE IF NOT EXISTS Employee_Salary_Audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT NOT NULL,  
    old_salary DECIMAL(10,2),  
    new_salary DECIMAL(10,2),  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    action VARCHAR(30)  
);
```

2. SALARY VALIDATION TRIGGER

Reject salary values outside the allowed range of 30,000 to 1,50,000.

```
DELIMITER //
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be between 30000 and
150000';
    END IF;
END//
DELIMITER
```

3. SALARY UPDATE VALIDATION TRIGGER

Validate salary changes before an existing Employee row is updated.

```
UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;
```

```
Valid Update:
UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;
```

4. SALARY AUDIT TRIGGER

Store the old and new salary values after a successful salary update.

```
DELIMITER //
CREATE TRIGGER audit_salary_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Employee_Salary_Audit
        (emp_id, old_salary, new_salary, action)
        VALUES
        (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATE');
    END IF;
END//
DELIMITER ;
```

```

CODE:
DELIMITER //
-- Test salary update
UPDATE Employee
SET salary = 51000
WHERE emp_id = 31;

-- Display salary audit record
SELECT audit_id, emp_id, old_salary, new_salary,
       changed_at, action
FROM Employee_Salary_Audit
ORDER BY audit_id;
DELIMITER

```

5. STORED PROCEDURE – TRANSFER EMPLOYEE

Transfer an employee to a new department after validating both IDs. The transaction is rolled back if an SQL exception occurs.

```

DELIMITER //
CREATE PROCEDURE transfer_employee (
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    DECLARE v_old_dept_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;

    SELECT COUNT(*) INTO v_emp_count FROM Employee WHERE emp_id =
p_emp_id;
    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Employee does
not exist';
    END IF;

    SELECT COUNT(*) INTO v_dept_count FROM Department WHERE
dept_id = p_new_dept_id;
    IF v_dept_count = 0 THEN

```

```

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Department
does not exist';
    END IF;

    SELECT dept_id INTO v_old_dept_id FROM Employee WHERE emp_id
= p_emp_id;
    IF v_old_dept_id = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Employee is
already in this department';
    END IF;

    UPDATE Employee SET dept_id = p_new_dept_id WHERE emp_id =
p_emp_id;
    COMMIT;
END//
DELIMITER ;

```

6. TEST SUCCESSFUL TRANSFER

Transfer employee 6 from Development to Sales.

```
CALL transfer_employee(6, 4);
```

```

SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 6;

```

```

154 BEGIN
155
156 SELECT COUNT(*) INTO v_dept_count
157 FROM Department
158 WHERE dept_id = p_new_dept_id;
159
160 IF v_dept_count = 0 THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Department does not exist';
163 END IF;
164
165 SELECT dept_id INTO v_old_dept_id
166 FROM Employee
167 WHERE emp_id = p_emp_id;
168
169 IF v_old_dept_id = p_new_dept_id THEN
170     SIGNAL SQLSTATE '45000'
171     SET MESSAGE_TEXT = 'Employee is already in this department';
172 END IF;
173
174 UPDATE Employee
175 SET dept_id = p_new_dept_id
176 WHERE emp_id = p_emp_id;
177
178 COMMIT;
179
180 END//
181 DELIMITER ;
182
183 CALL transfer_employee(6, 4);
184
185 -- Same department
186 CALL transfer_employee(6, 4);
187
188 SELECT e.emp_id, e.emp_name,
189        d.dept_name
190 FROM Employee e
191 JOIN Department d ON e.dept_id = d.dept_id
192 WHERE e.emp_id = 6;
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216

```

Output:

```

ERROR 1644 (45000) at line 287: Employee is already in this department

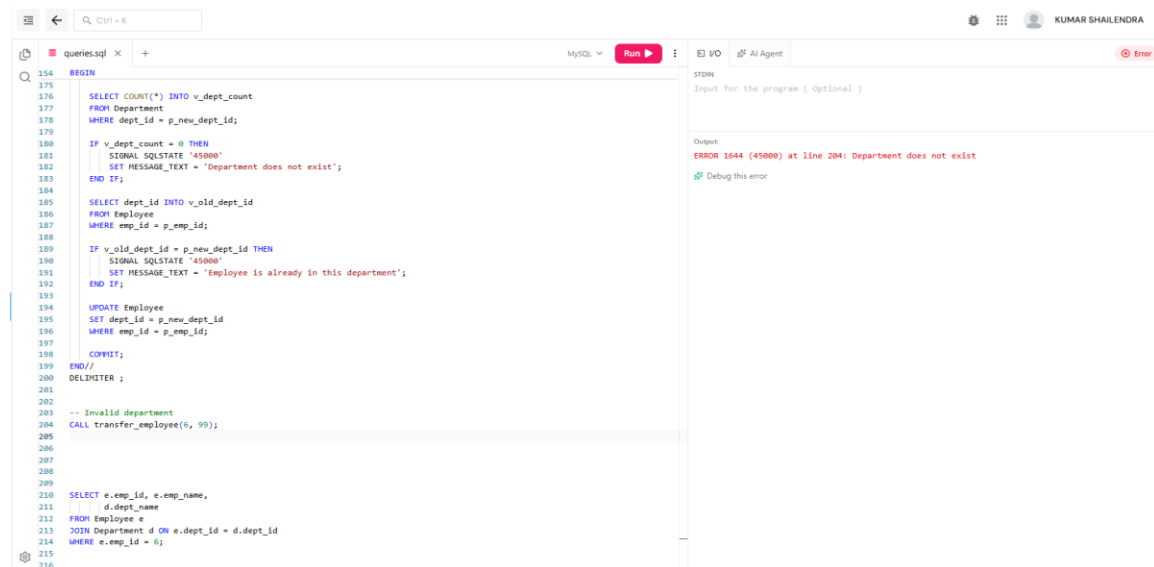
```

7. TEST EDGE CASES

```
-- Invalid employee  
CALL transfer_employee(99, 4);
```

```
-- Invalid department  
CALL transfer_employee(6, 99);
```

```
-- Same department  
CALL transfer_employee(6, 4);
```



The screenshot shows a MySQL IDE interface. On the left, a SQL script is displayed in a text editor. The script includes a procedure to transfer an employee, with comments indicating test cases for invalid employee ID, invalid department ID, and the same department. The script is as follows:

```
154 BEGIN  
155  
156 SELECT COUNT(*) INTO v_dept_count  
157 FROM Department  
158 WHERE dept_id = p_new_dept_id;  
159  
160 IF v_dept_count = 0 THEN  
161 SIGNAL SQLSTATE '45000'  
162 SET MESSAGE_TEXT = 'Department does not exist';  
163 END IF;  
164  
165 SELECT dept_id INTO v_old_dept_id  
166 FROM Employee  
167 WHERE emp_id = p_emp_id;  
168  
169 IF v_old_dept_id = p_new_dept_id THEN  
170 SIGNAL SQLSTATE '45000'  
171 SET MESSAGE_TEXT = 'Employee is already in this department';  
172 END IF;  
173  
174 UPDATE Employee  
175 SET dept_id = p_new_dept_id  
176 WHERE emp_id = p_emp_id;  
177  
178 COMMIT;  
179 END//  
180 DELIMITER ;  
181  
182 -- Invalid department  
183 CALL transfer_employee(6, 99);  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

On the right, the execution output is shown. It indicates an error at line 204: "ERROR 1644 (45000) at line 204: Department does not exist". A "Debug this error" link is provided below the error message.

8. TEST SALARY VALIDATION

Attempt to insert or update a salary outside the allowed range.

```
UPDATE Employee  
SET salary = 20000  
WHERE emp_id = 6;
```

```
UPDATE Employee  
SET salary = 85000  
WHERE emp_id = 6;
```

Expected Output:

ERROR 1644 (45000): Salary must be between 30000 and 150000

The screenshot shows a MySQL IDE window with a file named 'queries.sql'. The code is a PL/SQL procedure that takes two parameters: p_emp_id and p_new_dept_id. It first checks if the department exists by counting rows in the Department table. If the count is zero, it raises an error 'Department does not exist'. Then, it checks if the employee is already in the new department by querying the Employee table. If the employee is found, it raises an error 'Employee is already in this department'. If both checks pass, it updates the employee's department in the Employee table and commits the transaction. The procedure is then called with p_emp_id = 99 and p_new_dept_id = 4. The output pane on the right shows an error: 'ERROR 1644 (45000) at line 205: Employee does not exist'. Below the error message is a 'Debug this error' link.

```
154 BEGIN
155
156   SELECT COUNT(*) INTO v_dept_count
157   FROM Department
158   WHERE dept_id = p_new_dept_id;
159
160   IF v_dept_count = 0 THEN
161     SIGNAL SQLSTATE '45000'
162     SET MESSAGE_TEXT = 'Department does not exist';
163   END IF;
164
165   SELECT dept_id INTO v_old_dept_id
166   FROM Employee
167   WHERE emp_id = p_emp_id;
168
169   IF v_old_dept_id = p_new_dept_id THEN
170     SIGNAL SQLSTATE '45000'
171     SET MESSAGE_TEXT = 'Employee is already in this department';
172   END IF;
173
174   UPDATE Employee
175   SET dept_id = p_new_dept_id
176   WHERE emp_id = p_emp_id;
177
178   COMMIT;
179
180 END//
181 DELIMITER ;
182
183
184 -- Invalid employee
185 CALL transfer_employee(99, 4);
186
187
188
189
190 SELECT e.emp_id, e.emp_name,
191        d.dept_name
192 FROM Employee e
193 JOIN Department d ON e.dept_id = d.dept_id
194 WHERE e.emp_id = 6;
195
196
```

Output:

```
ERROR 1644 (45000) at line 205: Employee does not exist
Debug this error
```

9. TEST SALARY AUDIT LOG

Display the audit entries created by successful salary changes.

```
SELECT audit_id, emp_id, old_salary, new_salary,
       changed_at, action
FROM Employee_Salary_Audit
ORDER BY audit_id;
```

RESULT

The transfer_employee procedure was created with validation, transaction control and error handling. Salary validation and audit triggers were implemented and edge cases were tested.

CONCLUSION

Stored procedures centralize controlled database operations, while triggers automatically enforce validation rules and maintain an audit trail for important changes.